Plotly.js crosshair performance: bypassing internal event handlers

The CPU spikes you're experiencing are caused by Plotly's internal hover computation pipeline, which iterates through all data points on every mousemove regardless of your `hovermode: false` setting. The solution is to suppress this computation using `plotly_beforehover` returning `false`, combined with a DOM overlay architecture for your custom crosshair. This approach reduces update latency from **10-30ms to under 1ms** per frame.

The `plotly_beforehover` event is your primary weapon

The most powerful documented method for eliminating Plotly's internal CPU overhead is the `plotly_beforehover` event callback. When this handler returns `false`, Plotly completely skips its hover calculation pipeline— `Plotly` the exact code path causing your performance issues.

```javascript
const plotDiv = document.getElementById('myDiv');

// Critical: prevents ALL hover computation
plotDiv.on('plotly_beforehover', () => false);

// Combined with these layout settings
Plotly.relayout(plotDiv, {
  hovermode: false,
  dragmode: false
});
```

This approach stops Plotly from running distance calculations against every data point during mousemove events. The internal hover system (in `src/components/fx/hover.js`) normally iterates through all points without spatial indexing—a linear O(n) operation that becomes expensive with large datasets. GitHub issue #5641 documents how hover "loops over every data point" and suggests binary search for monotonous x-axis data.

Setting `hovermode: false` alone is insufficient because Plotly still processes internal mousemove handlers in `dragbox.js` for drag preparation. The `plotly_beforehover` callback intercepts this earlier in the pipeline.

Trace-level suppression with `hoverinfo: 'skip'`

For additional optimization, configure individual traces to be completely excluded from hover calculations:

```javascript
const traces = data.map(trace => ({
  ...trace,
  hoverinfo: 'skip'  // Blocks hover labels AND hover events
}));
```

The `'skip'` value differs from `'none'` in an important way: `'skip'` excludes the trace entirely from hover calculations, while `'none'` still includes the trace in event data but hides labels. GitHub PR #5854 specifically added `'skip'` to improve performance for auxiliary traces that don't need interactivity.

Why `staticPlot: true` may be too aggressive

The `staticPlot` configuration option completely disables all event listeners and interactivity:

```javascript
Plotly.newPlot('myDiv', data, layout, { staticPlot: true });
```

This eliminates **all** internal mouse tracking, removes the invisible DOM elements used for interaction capture, and changes antialiasing settings for print optimization. However, it also prevents you from receiving any Plotly events whatsoever—including `plotly_click` or axis coordinate updates you might need.

Use `staticPlot: true` only if you're building an entirely custom interaction layer and don't need any Plotly event integration.

The transparent overlay architecture works well

Placing a transparent div on top of the Plotly chart to intercept mouse events is a **proven pattern used by D3.js, ECharts, and other charting libraries.** D3 Graph Gallery It won't cause issues with Plotly's rendering when implemented correctly.

```javascript
const gd = document.getElementById('myDiv');
const layout = gd._fullLayout;

const overlay = document.createElement('div');
overlay.style.cssText = `
  position: absolute;
  top: ${layout.margin.t}px;
  left: ${layout.margin.l}px;
  width: ${layout.xaxis._length}px;
  height: ${layout.yaxis._length}px;
  pointer-events: none;
  contain: strict;
`;
gd.style.position = 'relative';
gd.appendChild(overlay);
```

For your Shift+mousemove conditional crosshair, handle events on the plot container (not the overlay) and toggle crosshair visibility based on modifier key state. The overlay with `pointer-events: none` purely displays the crosshair lines without interfering with Plotly's zoom/pan when the crosshair is inactive.

Click-through considerations: Use `pointer-events: auto` selectively on interactive overlay elements like tooltips or buttons, while keeping the crosshair lines themselves at `pointer-events: none`.

Coordinate conversion uses internal but stable APIs

Plotly exposes coordinate conversion functions on the axis objects that are widely used despite being undocumented:

```javascript
const xaxis = gd._fullLayout.xaxis;
const yaxis = gd._fullLayout.yaxis;

// Pixel to data value
const dataX = xaxis.p2l(pixelX);  // "pixel to linear"
const dataY = yaxis.p2l(pixelY);

// Data value to pixel (for positioning markers)
const pixelX = xaxis.l2p(dataX);  // "linear to pixel"

// For log/date axes, use d2p/p2d variants
const pixelX = xaxis.d2p(dataX);  // "data to pixel"
```

Listen for `plotly_relayout` events to update your overlay dimensions when the chart resizes or axis ranges change.

Built-in spike lines have significant limitations

Plotly's native spike lines (`showspikes: true`) are optimized but restrictive for custom crosshair needs:

```javascript
layout: {
  xaxis: {
    showspikes: true,
    spikemode: 'across+marker',
    spikesnap: 'cursor'  // Follows cursor freely
  }
}
```

Key limitations that likely rule this out for your use case:

- No support for Shift-key modifier to conditionally show crosshairs
- Cannot display custom sidebar panels or additional markers
- `spikesnap: 'cursor'` only works with certain hovermodes
- Cannot have both 'x unified' and 'y unified' hovermodes simultaneously `Plotly`
- Limited styling options (no gradients, shadows, or complex patterns)

The spike lines are handled internally within Plotly's optimized rendering pipeline, so they're performant, but the customization constraints make the DOM overlay approach superior for your requirements.

Never use `Plotly.relayout()` for 60fps shape updates

Updating crosshair position via Plotly shapes is approximately **30x slower** than direct DOM manipulation:

Approach	Update latency	Notes
`Plotly.relayout()` shapes	10-30ms	Triggers internal recalculations
`Plotly.react()`	~10-30ms	Slightly more efficient but similar
Direct DOM manipulation	<1ms	Bypasses Plotly entirely

GitHub issue #5522 explicitly states: *"These APIs weren't designed for high FPS interactions, each call can trigger a lot of recalculations, whether they're needed or not."* Issue #5998 documents this exact problem—creating vertical line crosshairs via shapes causes noticeable lag.

Transform positioning outperforms top/left by avoiding layout

Always position crosshair elements using CSS transforms rather than top/left properties:

```javascript
// ❌ Triggers layout recalculation on every update
crosshair.style.top = `${y}px`;
crosshair.style.left = `${x}px`;

// ✅ GPU-accelerated, no layout thrashing
crosshair.style.transform = `translate(${x}px, ${y}px)`;
```

Transforms are composited on the GPU without triggering browser reflow. Combined with `will-change: transform` on the crosshair element, this enables smooth sub-pixel positioning at 60fps.

```css
.crosshair-line {
  position: absolute;
  top: 0;
  left: 0;
  will-change: transform;
  contain: layout paint;
  pointer-events: none;
}
```

The `contain: layout paint` declaration tells the browser this element's internal changes won't affect outside layout, allowing the browser to skip expensive layout calculations.

---

requestAnimationFrame beats throttling for visual updates

Your current 15-30fps throttling via `setTimeout` introduces perceptible lag. Switch to requestAnimationFrame with a "last value wins" pattern:

```javascript
function createRAFThrottle() {
  let pending = null;
  return (callback) => {
    if (!pending) {
      requestAnimationFrame(() => {
        const cb = pending;
        pending = null;
        cb();
      });
    }
    pending = callback;
  };
}

const throttledUpdate = createRAFThrottle();

plotDiv.addEventListener('mousemove', (e) => {
  if (!shiftKeyHeld) return;

  throttledUpdate(() => {
    updateCrosshairPosition(e.clientX, e.clientY);
  });
});
```

This pattern only executes the final callback before each frame paint, discarding intermediate mousemove events that would be wasted work. nolanlawson It naturally aligns with the browser's ~**60fps refresh rate** (~16.6ms intervals) and adapts to the device's actual rendering capability.

WebGL mode doesn't significantly change event performance

Using `scattergl` instead of `scatter` won't fix your mousemove performance issues. WebGL traces dramatically improve **rendering** performance for large datasets (100k+ points), but mouse events are still handled at the container level regardless of renderer.

The event handling architecture is the same: events hit the container, Plotly computes hover data by searching through points, and triggers callbacks. The WebGL vs SVG choice affects how quickly the points **draw**, not how quickly Plotly **finds** the nearest point during hover.

Where WebGL helps is if your current slowdown includes significant redraw time on the chart itself during crosshair updates—but based on your description (CPU spikes even with custom drawing disabled), the bottleneck is clearly in Plotly's event processing, not rendering.

GitHub issues confirm your diagnosis

Your diagnosis that "Plotly's internal mousemove handling is the culprit" is confirmed by multiple GitHub issues:

- **Issue #503**: Documents that `maindrag.onmousemove` takes ~3ms per call, compared to 0.6ms in dygraphs
- **Issue #3826**: Hover performance degrades with trace count—1M points in one trace is 10x faster than across 100 traces
- **Issue #5641**: Profiling shows hover loops over every data point without spatial indexing
- **Issue #5998**: Exact same use case—vertical line crosshair lag on mousemove
- **PR #2623**: Implemented optimizations by stashing "hot" SVG selections and removing `selectAll` calls

The core architectural issue is that Plotly's hover system uses linear iteration rather than spatial data structures like quadtrees or binary search for sorted axes.

Complete recommended implementation

javascript

```
class PlotlyCrosshair {
  constructor(plotDiv, sidebar) {
    this.gd = plotDiv;
    this.sidebar = sidebar;
    this.active = false;

    this.suppressPlotlyHover();
    this.createOverlay();
    this.bindEvents();
  }

  suppressPlotlyHover() {
    this.gd.on('plotly_beforehover', () => false);
    Plotly.relayout(this.gd, {
      hovermode: false,
      dragmode: false
    });
  }

  createOverlay() {
    const layout = this.gd._fullLayout;

    this.overlay = document.createElement('div');
    this.overlay.innerHTML = `
      <div class="crosshair-v"></div>
      <div class="crosshair-h"></div>
    `;
    this.overlay.style.cssText = `
      position: absolute;
      top: ${layout.margin.t}px;
      left: ${layout.margin.l}px;
      width: ${layout.xaxis._length}px;
      height: ${layout.yaxis._length}px;
      pointer-events: none;
      contain: strict;
      display: none;
    `;
    this.gd.appendChild(this.overlay);

    this.vLine = this.overlay.querySelector('.crosshair-v');
    this.hLine = this.overlay.querySelector('.crosshair-h');

    // Style crosshair lines
    const lineStyle = `position:absolute;background:rgba(100,100,100,0.5);
      pointer-events:none;will-change:transform;`;
    this.vLine.style.cssText = lineStyle + 'width:1px;height:100%;top:0;';
    this.hLine.style.cssText = lineStyle + 'height:1px;width:100%;left:0;';
  }
```

```
}

bindEvents() {
  document.addEventListener('keydown', e => {
    if (e.key === 'Shift') this.active = true;
  });
  document.addEventListener('keyup', e => {
    if (e.key === 'Shift') {
      this.active = false;
      this.overlay.style.display = 'none';
    }
  });

  const throttle = this.createRAFThrottle();
  this.gd.addEventListener('mousemove', e => {
    if (!this.active) return;
    throttle(() => this.update(e));
  });

  this.gd.on('plotly_relayout', () => this.updateDimensions());
}

createRAFThrottle() {
  let pending = null;
  return cb => {
    if (!pending) {
      requestAnimationFrame(() => {
        const fn = pending;
        pending = null;
        fn();
      });
    }
    pending = cb;
  };
}

update(e) {
  const layout = this.gd._fullLayout;
  const rect = this.gd.getBoundingClientRect();
  const x = e.clientX - rect.left - layout.margin.l;
  const y = e.clientY - rect.top - layout.margin.t;

  // Boundary check
  if (x < 0 || x > layout.xaxis._length ||
    y < 0 || y > layout.yaxis._length) {
    return;
  }

  // GPU-accelerated positioning
  this.vLine.style.transform = `translateX(${x}px)`;
```

```
        this.hLine.style.transform = `translateY(${y}px)`;
        this.overlay.style.display = 'block';

        // Update sidebar with data coordinates
        const dataX = layout.xaxis.p2l(x);
        const dataY = layout.yaxis.p2l(y);
        this.updateSidebar(dataX, dataY);
    }

    updateSidebar(x, y) {
        this.sidebar.textContent = `X: ${x.toFixed(4)}, Y: ${y.toFixed(4)}`;
    }

    updateDimensions() {
        const layout = this.gd._fullLayout;
        this.overlay.style.top = `${layout.margin.t}px`;
        this.overlay.style.left = `${layout.margin.l}px`;
        this.overlay.style.width = `${layout.xaxis._length}px`;
        this.overlay.style.height = `${layout.yaxis._length}px`;
    }
}
```

Conclusion

The key insight is that **disabling visible hover features doesn't disable Plotly's internal hover computation**. The `plotly_beforehover` event returning `false` is the critical mechanism for actually bypassing the expensive point-finding loop. Combined with a DOM overlay architecture using CSS transforms and requestAnimationFrame, you can achieve smooth 60fps crosshair tracking regardless of dataset size. Avoid `Plotly.relayout()` for any per-frame updates—it's designed for configuration changes, not animation.